Predicting Bazel Build Times with Machine Learning

UE24CS352A Machine Learning, Mini-Project | Team: [Name] ([SRN]) & [Name] ([SRN]) | PES University, Dept. of CSE (AI-ML)

1. Problem statement

Bazel is Google's open-source build system. Because it is **incremental**, a new commit only re-executes the build actions whose inputs changed, so build cost varies from under a second to tens of seconds. The task, taken from Nadeem & Raza (Stanford CS229), is to predict the **CPU time of the build that follows a commit** using only information available before building: which files the commit changed. Such a predictor lets CI schedulers batch or prioritise builds and warns developers that a change will be expensive. Our goal was to reproduce the paper, audit it, and improve it.

2. Dataset

500 consecutive commits of `bazelbuild/bazel` (May–June 2021). The authors checked out each commit with `git checkout HEAD~1`, ran `bazel build //src:bazel-dev` and read *cpuTimeInMs* from the Build Event Protocol (BEP) JSON. Target: CPU ms, median 1.8 s, mean 4.3 s, max 44.9 s, heavily right-skewed. Paper inputs: the single most common path prefix (11 values) and file type (5 values) per commit. **Our enrichment:** the stored commit IDs match Bazel's first-parent history one-to-one, so we recovered the **full list of changed files and line counts** for all 500 commits, and extracted *actionsExecuted* from the BEP files. Split as in the paper: builds 1–300 train, 301–400 validation, 401–500 test.

3. Approach

- **Reproduce.** Linear regression trained by mini-batch SGD (lr 0.01, batch 16, our numpy implementation) on one-hot prefix + type and their 55-way **feature cross**; compared with a predict-the-mean baseline.
- **Audit.** Read the original code and data. Found (i) `test_model()` builds a new *untrained* regressor, so all test predictions are 0 (RMSE 7,459 ms, the flat test line in the paper); (ii) substring matching: `'.c'` matches .cc/.css, `'.h'` matches .html, commits with no listed type default to JAVA, and `'tools/'` matches every `com/google/devtools/` path (144 of 171 'tools/' labels have no file in tools/).
- **Fix the alignment.** Walking history backwards with a warm cache means build *i* re-does the work caused by the difference between commit *i* and the previously built commit *i–1*: the files changed **by commit i–1**. The paper used commit *i*'s own diff. We shift features by one.
- **Richer features.** Files are mapped to a source area (src/main/java, src/test, site, …, by longest prefix) and a type (by real extension). Features: per-area and per-type counts, log #files, log lines added/deleted, flags *touches src/main*, *only tests/docs*, *touches BUILD/.bzl*: 32 in all.
- **Size and dependency features.** For the changed files: total and largest size in bytes and number of imports / `#includes`, overall and for Java and C/C++ (8 more features), read from git in the state *before* the commit, the version the backwards walk compiles.
- **Proxy check.** Two file-based cost estimates from a companion project that mined all 47k Bazel commits (fixed language/directory weights; and compile times fitted on measured *javac*/*g++* runs) are compared with the real CPU time, and tried as features.
- **Models.** Ridge regression, random forest and gradient boosting (Huber loss) on **log CPU time**; feature set and model selected together on validation RMSLE (gradient boosting with size features won); final model retrained on train+validation and evaluated once on test. Because 100 test builds are few, feature sets are also compared by paired 5-fold CV repeated 20 times. The cold-cache first build (44.9 s) is excluded from training.

4. Implementation

Python 3 with pandas, scikit-learn and matplotlib (the original TensorFlow-1 Estimator code no longer runs on TF 2). `scripts/enrich_from_git.py` rebuilds the diff data and `scripts/size_features.py` the size features from a Bazel clone; `src/features.py` holds path classification and the alignment; `src/models.py` the SGD regressor and model zoo; `src/train.py` runs every experiment in about 4.5 min and writes metrics.json, figures and the model; `src/predict.py` is the demo CLI (changed files, a real commit, or a replayed dataset row → predicted CPU time and a cheap/medium/expensive label).

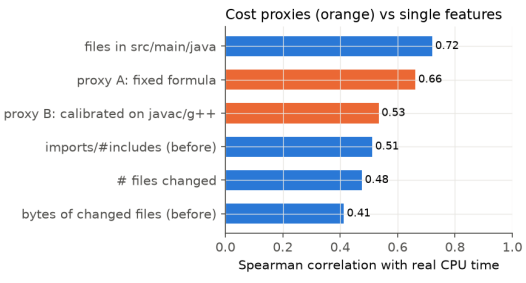


Fig A. Spearman correlation with real CPU time: cost proxies (orange) vs single features.

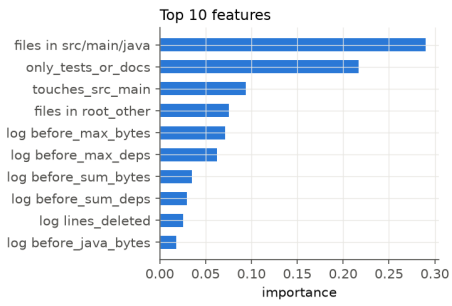


Fig B. Final model, gradient-boosting feature importance.

5. Results

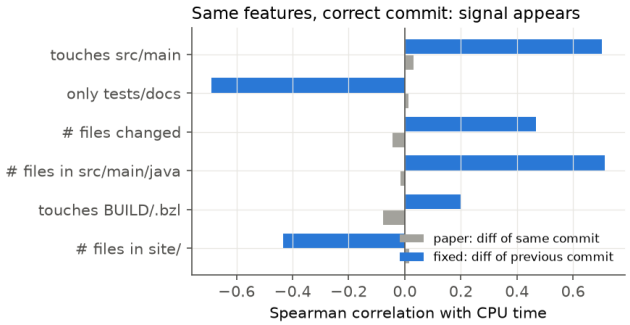


Fig 1. Spearman correlation of features with CPU time, before and after the alignment fix.

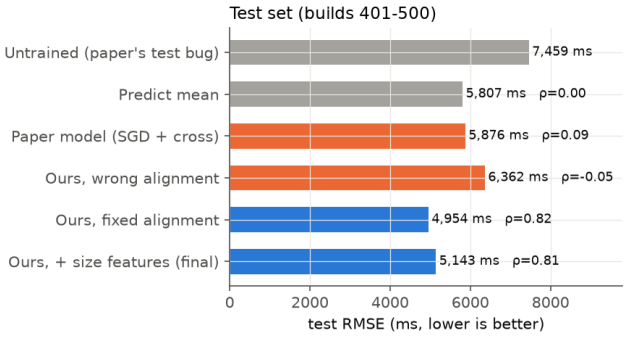


Fig 2. Test RMSE; ρ = Spearman rank correlation.

Model (test, builds 401–500)	RMSE ms	MAE ms	RMSLE	Spearman	AUC >5 s
Predict training mean	5,807	4,352	1.28	0.00	0.50
Paper model reproduced (SGD + cross)	5,876	4,394	1.28	0.09	0.55
Diff features, paper alignment	6,362	3,822	1.20	-0.05	0.45
Diff features, fixed alignment	4,954	2,744	0.69	0.82	0.85
+ size features (final, chosen on validation)	**5,143**	**2,811**	**0.73**	**0.81**	**0.83**

The reproduced paper model matches the paper's training RMSE (about 5.4 to 5.7 s) but is **no better than predicting the mean** (ρ = 0.09). Our richer features alone do not help (ρ = -0.05); the signal appears only after the alignment fix (*touches src/main*: ρ 0.03 → 0.70). The final model cuts MAE by 36% and identifies builds over 5 s with AUC 0.83. On these 100 test builds it is slightly behind the diff-only model, well inside the fold-to-fold spread; 5-fold CV over all 499 incremental builds gives RMSE 4,210 ± 701 ms, ρ 0.80 (diff features only: 4,709 ms, 0.78; paper's features: ρ 0.00). On identical folds the size features lower RMSLE by 0.040 in 95% of 100 folds (Fig 4). The BEP data explains why paths work: CPU time correlates 0.92 (Spearman) with the number of actions Bazel re-executed, and 149 builds ran a single action (docs/test-only changes).

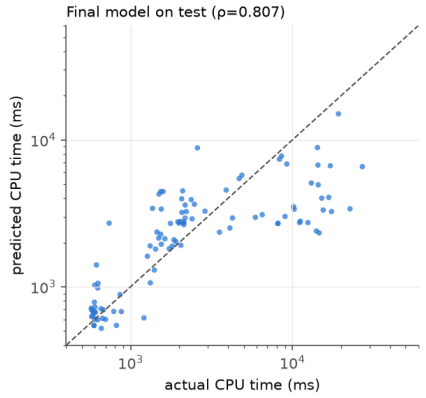


Fig 3. Final model, test builds (log axes). Cheap builds are nailed; the largest are ranked correctly but under-predicted.

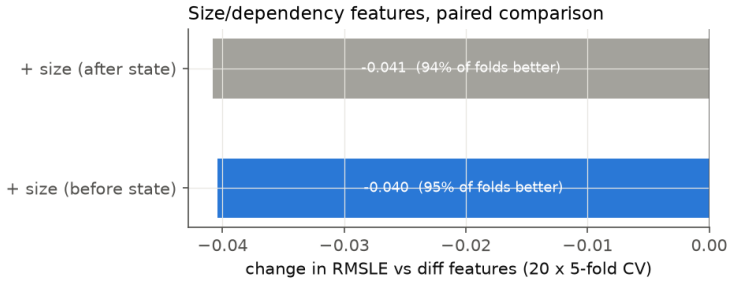


Fig 4. Paired 20 x 5-fold CV: change in RMSLE when size features are added (negative is better). Before and after state give the same gain.

6. Conclusions

- Build cost is driven by **how much of the dependency graph a change invalidates**; which files changed (core `src/main/java` vs tests or docs) is a strong proxy for that, and how large the changed files are and how many dependencies they declare adds a consistent gain.
- Hand-built cost formulas are weaker than learned features: the two proxies reach ρ 0.66 and 0.53; measured single-file compile times do not transfer to what Bazel re-runs.
- Data provenance mattered more than the model: an off-by-one between features and labels erased all signal. Always compare against a trivial baseline and look at rank metrics, not RMSE alone, when the target is skewed.
- Limitations: 500 builds, one machine and one target; 100 test builds are too few to separate models a few percent apart; the model compresses the extreme tail (> 15 s builds are ranked correctly but under-predicted). Next step: reverse-dependency counts from `bazel query 'rdeps(...)'` as features, and more commits.

Reference: M. U. Nadeem, S. Raza, "Predicting Bazel Build Times Using Machine Learning", Stanford CS229; code and data: github.com/umar-nadeem/CS229-Final-Project.